COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S2
DATE PERFORMED:	9/2/2026	**DATE SUBMITTED:**	9/2/2026
NAME:	Fernandez, Cedric S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 6.1	**Searching Techniques**

1. PROCEDURE OUTPUT

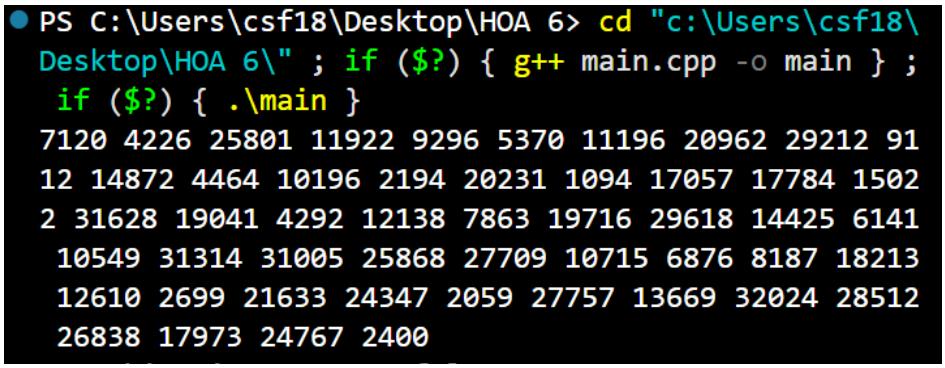
SCREEN SHOT	
OBSERVATIONS	The code was able to initialize a random dataset with the use of srand with the help of the for loop so that it executes until it's no longer less than the max_size while also printing every value the srand has given

Table 6-1. Data Generated and Observations.

CODE

main.cc

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

//GENERATE RANDOM VALUES

    int dataset[max_size];
    srand(time(0));
```

```
    for(int i = 0; i <max_size ; i++){
         dataset[i] = rand();
         std::cout<< dataset[i] <<" ";
     }
     std::cout<<std::endl;

    std::cout<<"What number to find: ";
    int choice;

    std::cin>>choice;
    linearSearchAR(max_size,choice, dataset);
```

searchingAlgo.h

```
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){

    //while loop or for loop
    int i= 0;

    while(i < size){
        if(dataFind == array[i]){
            std::cout<<"Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout<<"Searching is unsuccessful\n";
}
```

OUTPUT

Number not found

```
20143 8327 30484 31713 19979 2065 4579 12398 13754 1082 14846 16282 23040
 26959 16207 4921 12462 31720 8666 461 29323 7360 24132 30973 27640 26308
 25490 11330 16596 29546 5684 7140 10142 14660 1090 2085 24537 14549 1203
7 9403 26549 27555 3513 8324 13547 10637 11665 21878 29929 30815
What number to find: 10
Searching is unsuccessful
```

	Number found 20179 28255 30382 1497 2367 12491 6048 12966 31542 31757 13219 16766 4424 12360 30945 31800 12548 5022 28804 29856 23821 13811 25455 9829 24433 16 883 30508 28723 28215 24406 5926 10268 32311 2658 32281 20284 14383 1202 29638 10397 17426 17207 4843 14168 24227 32124 11122 26621 27971 16942 What number to find: 9829 Searching is successful
OBSERVATIONS	The program generates a list of 50 random values and displays them all after that it will ask the user on what number they would like to search, then with the while loop it will do a linear search by checking every item in the array one by one until it finds the one that is a match to the users input if it finds it it prints searching successful if not searching unsuccessful

Table 6-2a. Linear Search for Arrays

CODE

main.cc

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "searchingAlgo.h"
#include "node.h"

const int max_size = 50;

int main() {
    Node<char> *char1 = new_node('C');
    Node<char> *char2 = new_node('E');
    Node<char> *char3 = new_node('D');
    Node<char> *char4 = new_node('R');
    Node<char> *char5 = new_node('I');
    Node<char> *char6 = new_node('C');

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;

    linearSearchLL(char1, 'R');
}
```

node.h

```
#ifndef NODE_H
```

```
#define NODE_H

template<typename T>
class Node{
    public:
    T data;
    Node* next;
};

template<typename T>
Node<T>* new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
```

searchingAlgo.h

```
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the current node is not NULL

    while(nodes != nullptr){

        //check if the item = current node data
        if(nodes->data == dataFind){
            std::cout << "The data: " << nodes->data << "
was found successfully.\n";
            return;
        }
        // move on to the next node
        nodes = nodes->next;

    }
```

	``
	    //display unsuccessful
	    std::cout << "Searching is unsuccessful.\n";
	}
	
	#endif
	``

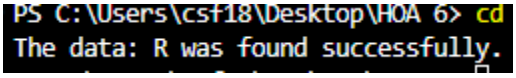
OUTPUT	
OBSERVATION	The following first creates a linked list storing the letters of my name, then while the while loop the linear search function will go through every node starting from head to tail comparing everything to the value that it is looking for which is R then after it is found it will print alongside the data was found successfully.

Table 6-2b. Linear Search for Linked List

CODE

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "searchingAlgo.h"
#include "node.h"

const int max_size = 50;

int main() {
int newArr[max_size];
for(int i = 0; i < max_size; i++){
    newArr[i] = i;
}

binarySearchArr(newArr, max_size, 100); // 0-49 to be
found
}
```

searchingAlgo.h

```
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H
```

	``
	template <typename T>
	void binarySearchArr(T data[], int arrSize, T dataFind){
	    int low = 0, high = arrSize - 1, middle;
	    while(low <= high){
	        middle = (int)((low+high)/2);
	        if(dataFind == data[middle]){
	            std::cout<<"the data was found\n";
	            return;
	        }
	        else if(dataFind < data[middle]){
	            high = middle -1;
	        }
	        else{
	            low = middle+1;
	        }
	    }
	    std::cout<<"the data was not found\n";
	}
	#endif
	``

OUTPUT	**Number not found (numbers that are not 0-49)** the data was not found **Number found (numbers that are 0-49)** the data was found
OBSERVATION	The programs creates a sorted array using for loop that in which contains 0 - 49 then using the binary search function it divides the search range by half every step then if the midpoint element matches the value being searched for then it prints the data was found if not it keeps adjusting to the low and high until it can't anymore and it prints data was not found.

Table 6-2b. Linear Search for Arrays

CODE	**main.cc** ``` #include <iostream> #include <cstdlib> //for generating random integers #include <time.h> //will be used for our seeding function #include "searchingAlgo.h" #include "node.h" ```

```
int main() {

    Node<char> *char1 = new_node('C');
    Node<char> *char2 = new_node('E');
    Node<char> *char3 = new_node('D');
    Node<char> *char4 = new_node('R');
    Node<char> *char5 = new_node('I');
    Node<char> *char6 = new_node('C');

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;
    binarySeachLL(char1, 'R');
}
```

searchingAlgo.h

```
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//middle function getter:

template <typename T>

Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    //continue until fast reaches the end or the last ode
of the current search range

    while(fast != last && fast->next != last){

        //move slow one step
```

```
        slow = slow ->next;
        // move fast two steps
        fast = fast ->next->next;
    }

    return slow;
}

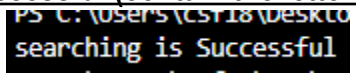
template <typename T>
void binarySeachLL(Node<T>* start, T dataFind){
    //step 1: set start to the head of the list and set
last to null
    Node<T>* last = nullptr;

    while(start != last){
        //Middle element is calculated using the two
pointers approach discussed above
        Node<T> *middle = getMiddle(start, last);

        //If the middle element is same as the key to be
searched, we return it.
        if(middle->data == dataFind){
            std::cout<< "searching is Successful\n";
            return;
        }
        //element
        else if(middle->data > dataFind){
            last = middle;
        }
        //Else if middle element is less than the key to
be searched, we have to search on the right
        //side of the singly linked list. So, we set start
pointer to the next element
        else{
            start = middle->next;
        }
    }
```

<table>
<tr><td></td><td>

```
    std::cout<<"Searching is unsucessful\n";
}
#endif
```

</td></tr>
<tr><td>OUTPUT</td><td><u>Searching successful (contain the letters C-E-D-R-I-C)</u>
 

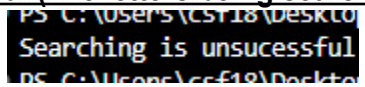
<u>Searching unsuccessful (The letters being searched is not C-E-D-R-I-C)</u>
 
</td></tr>
<tr><td>OBSERVATION</td><td>The following does a binary search for linked list that stores the characters C E D R I C we make use of the getmiddle function that makes use of the fast and slow pointer because a linked list does not have access to the index, then the binarysearchll function is the one that checks if the value matches the one we are looking for if not it adjust with the help of start and last in the else if and if to narrow the search until the target is found if its would it prints searching successful if not searching unsuccessful</td></tr>
</table>

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

Main.cc

```cpp
#include <iostream>
#include "searchingAlgoSup.h"

int main(){

    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;
    // For Array search
    int arrayComparisons = linearSearchAR(size, key, arr);
    std::cout << "Array search" << '\n';
    std::cout << "Comparisons needed: " << arrayComparisons << '\n';
    // For Linked list
    Node<int>* head = nullptr;
    Node<int>* tail = nullptr;

    for(int i = 0; i < size; i++){
        Node<int>* newNode = new_node(arr[i]);
        if(head == nullptr){
            head = newNode;
            tail = newNode;
        }
        else{
            tail->next = newNode;
            tail = newNode;
        }
    }
    int listComparisons = linearSearchLL(head, key);
    std::cout << "Linked list search" << '\n';
    std::cout << "Comparisons needed: " << listComparisons << '\n';
    deleteNodes(&head);
    return 0;
}
```

searchingAlgoSup.h

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

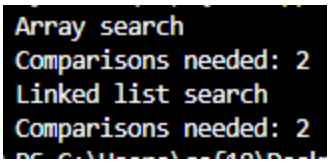
//changed to int so it could actually return a value instead of none
template <typename T>
int linearSearchAR(int size, T dataFind, T array[]){

    //added compare variable so we can use the following to give the number of comparissons for both
    //linearSearchAR and LinearSearchLL
    int i= 0;
    int compare = 0;

    while(i < size){
        compare++;
        if(dataFind == array[i]){
            return compare;
        }
        i++;
    }
    return compare;
}

template <typename T>
int linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the current node is not NULL
    int compare = 0;
    while(nodes != nullptr){
        compare++;
        if(nodes->data == dataFind){
            return compare;
        }
        // move on to the next node
        nodes = nodes->next;

    }
    return compare;
}
#endif
```

OUTPUT	
ANSWER	how many comparisons would be necessary to find the key '18'? **It takes 2 comparisons to find the key 18**

PROBLEM 1

CODE

Main.cc

```cpp
#include <iostream>
#include "searchingAlgoSup.h"

int main(){

    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int k = 18;

    int arrayCount = linearSearchAR(size, k, arr);
    std::cout << "Array" << std::endl;
    std::cout << "The number of occurrences of " << k
        << " is: " << arrayCount << std::endl;

    return 0;
}
```

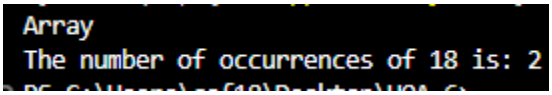
searchingAlgoSup.h

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
int linearSearchAR(int size, T dataFind, T array[]){

    int i = 0;
    int count = 0;
    while(i < size){
        if(dataFind == array[i]){
            count++;
        }
        i++;
    }
    return count;
}
#endif
```

OUTPUT	

PROBLEM 2

CODE

Main.cc

SupplementaryMain.cpp > ...

```cpp
#include <iostream>
#include "searchingAlgoSup.h"

int main(){
    int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int size = 10;
    int key = 8;
    binarySearchArr(arr, size, key);

    return 0;
}

```

searchingAlgoSup.h

```cpp
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind){

    int low = 0;
    int high = arrSize - 1;
    int middle;
    int iteration = 1;

    while(low <= high){
        middle = (low + high) / 2;
        std::cout << "Iteration " << iteration << std::endl;
        std::cout << "Low: " << low << std::endl;
        std::cout << "High: " << high << std::endl;
        std::cout << "Middle: " << middle << std::endl;
        std::cout << "Value at middle: " << data[middle] << std::endl;

        if(dataFind == data[middle]){
            std::cout << "The data was found!" << std::endl;
            return;
        }
        else if(dataFind < data[middle]){
            std::cout << "Key is smaller. Searching left." << std::endl;
            high = middle - 1;
        }
        else{
            std::cout << "Key is larger. Searching right." << std::endl;
            low = middle + 1;
        }

        iteration++;
        std::cout << std::endl;
    }
    std::cout << "The data was not found." << std::endl;
}
#endif
```

OUTPUT

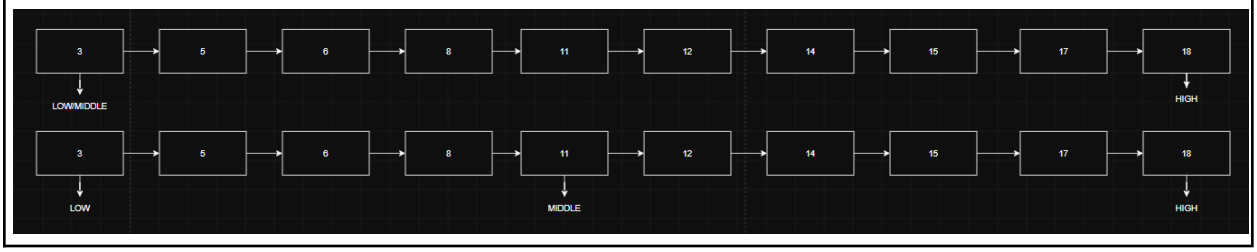
```
Iteration 1
Low: 0
High: 9
Middle: 4
Value at middle: 11
Key is smaller. Searching left.

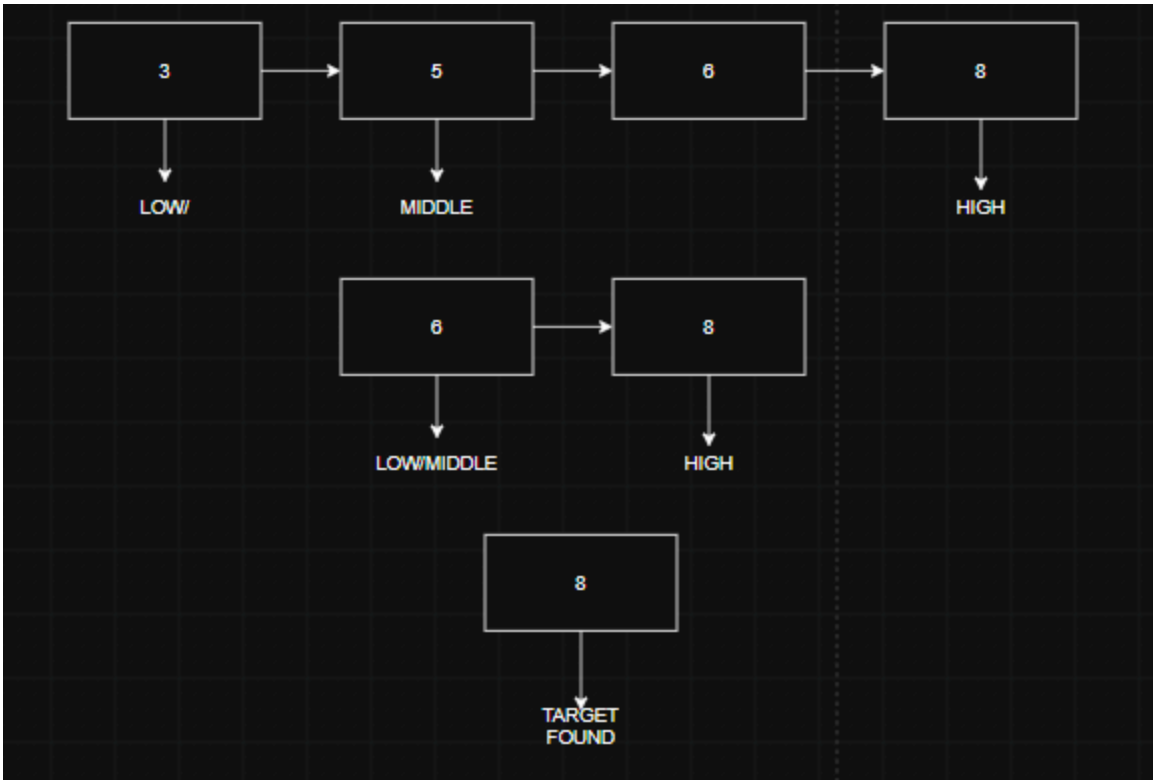
Iteration 2
Low: 0
High: 3
Middle: 1
Value at middle: 5
Key is larger. Searching right.

Iteration 3
Low: 2
High: 3
Middle: 2
Value at middle: 6
Key is larger. Searching right.

Iteration 4
Low: 3
High: 3
Middle: 3
Value at middle: 8
The data was found!
```

DIAGRAM





PROBLEM 3

CODE	Main.cc

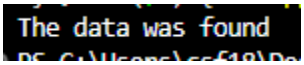
```cpp
#include <iostream>
#include "searchingAlgoSup.h"

int main(){

    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int dataFind = 8;

    binarySearchArr(data, 0, size - 1, dataFind);
    return 0;
}
```

	searchingAlgoSup.h
	`#include "node.h"`
	`#include <iostream>`
	
	`#ifndef SEARCHINGALGO_H`
	`#define SEARCHINGALGO_H`
	
	`template <typename T>`
	`void binarySearchArr(T data[], int low, int high, T dataFind){`
	
	`    if(low > high){`
	`        std::cout << "The data was not found\n";`
	`        return;`
	`    }`
	`    int middle = (low + high) / 2;`
	
	`    if(dataFind == data[middle]){`
	`        std::cout << "The data was found\n";`
	`        return;`
	`    }`
	`    else if(dataFind < data[middle]){`
	`        binarySearchArr(data, low, middle - 1, dataFind);`
	`    }`
	`    else{`
	`        binarySearchArr(data, middle + 1, high, dataFind);`
	`    }`
	`}`
	`#endif`

OUTPUT	

PROBLEM 4

CONCLUSION
In this activity, I applied linear and binary searching to arrays and linked lists. Linear search inspects items one by one, working well on any list structure. Binary search repeatedly divides the dataset in half, making it much faster for sorted arrays. Both algorithms are fundamental searching techniques in data structures. Although I ran into errors on Problems 3 and 4 I still managed to make everything work.
I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance: **Tools Used:** **Copilot, QuillBot** *Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.* **Purpose of Use:** **Grammar correction, Code Assistance, Explanation of concept** *Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

Extent of Use: **How to troubleshoot errors in a particular section that I had no clue on what I did wrong and also how to properly use certain conditions**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.